DSC 40B - Homework 04

Due: Wednesday, April 29

Write your solutions to the following problems by handwriting them on another piece of paper. Unless otherwise noted by the problem's instructions, show your work or provide some justification for your answer. Homeworks are due via Gradescope at 11:59 p.m.

Problem 1.

What is the *best* case time complexity of `quickselect`? Describe why and provide the input that yields this best case. For simplicity, you may assume that Quickselect always chooses the last element of the input array as the pivot (that is, the pivot is *not* chosen randomly).

Solution: The best case time complexity of quickselect is $\Theta(n)$, where n is the size of the input array.

In the best case, the element chosen as the first pivot is exactly the order statistic being queried so that no recursive calls are made. However, we still must perform the partition operation to count the number of elements to the left and right of the pivot, and this takes linear time.

Alternatively, another case which also leads to $\Theta(n)$ run time (and so is also a "best case") is when the chosen pivot is not the order statistic we are looking for, but it is the median (or close to it). Then there is still a recursive call on an array of size $n/2$, plus $\Theta(n)$ work for the partition. If the recursive calls also choose the median as their pivot, this gives a recurrence of $T(n) = T(n/2) + n$. Solving this gives a time complexity of $\Theta(n)$.

Problem 2.

Refer to the implementation of quickselect in lecture.

Suppose we use quickselect to select the minimum element of the array $A = [3, 2, 9, 0, 7, 5, 4, 8, 6, 1]$. Describe a sequence of partitions (which pivot index is chosen at each step) that results in a worst-case performance of quickselect.

Solution: For the worst case performance, the minimum element should be chosen as the pivot element at the very end. Moreover, we also want the array to be partitioned such that if arr has n elements, then partition on which the recursive call is made has n-1 elements. This can be done by choosing the maximum element in the array as the pivot. Hence, the pivot elements should be chosen in the order 9,8,7,6,5,4,3,2,1,0.

Problem 3.

An understanding of data structures and the time complexity of the various operations involving them is crucial when designing algorithms. Using the wrong data structure, or using the right data structure in the wrong way, can lead to an algorithm which is still correct, but significantly slower. We will see one example in this problem.

Another way of implementing the `merge` function used in `mergesort` is shown below:

```
def merge(left, right, arr):
    left.append(float('inf'))
    right.append(float('inf'))
    for ix in range(len(arr)):
        if left[0] < right[0]:
```

```
        arr[ix] = left.pop(0)
    else:
        arr[ix] = right.pop(0)
```

This code does essentially the same thing as the `merge` we saw in class: it merges two sorted lists `left` and `right` into a list `arr` such that the result is in sorted order. However, instead of using index variables to keep track of the front of `left` and `right`, this code uses `list.pop(0)` to remove the first element from the appropriate list.

a) Popping an element from the end of a Python list is quick and efficient, but popping the first element from a list requires that the remaining elements be shifted over to the left by one place. Shifting requires copying the elements, one by one, to their new location. What is the time complexity of `arr.pop(0)`, assuming that `arr` is a list of size n?

> **Solution:** $\Theta(n)$. Shifting one element to the left takes constant time. There are $n-1$ elements to shift, though, for a total time of $\Theta(n-1) = \Theta(n)$.

b) What is the time complexity of the new version of `merge`, assuming that (when they are passed to the function) `left` and `right` are lists of size $n/2$, and `arr` is a list of size n? Show your work by writing a formula for runtime of the function and its asymptotic runtime.

> **Solution:** First, we claim that the calls to `.pop(0)` will dominate the time complexity. Therefore, we calculate the total amount of time spent in these calls.
>
> Each iteration of the `for`-loop will result in one list element being popped, either from `left` or from `right`. The time it takes to perform this pop will depend on how many elements are in `left` or `right` at that moment. But this depends on which list was popped from in earlier iterations, which ultimately depends upon the contents of `list` and `right`. In short, there is no general way to say how large `left` or `right` are on the αth iteration.
>
> But that's OK. Instead, recognize that there will be exactly one call to `left.pop(0)` when `left` has $n/2+1$ elements (remember that we appended ∞ to the end). Because popping takes time linear in the number of elements, this will take time $c(n/2+1)$, where c is some constant. Some time later – perhaps on the next iteration, perhaps in five iterations – we will call `left.pop(0)` again. This will now take roughly $c \cdot n/2$ time, since there are now $n/2$ elements in the list. On the third call, whenever that will be, the pop will take roughly $c(n/2-1)$ time. And so on.
>
> In total, calls to `left.pop(0)` will take time:
>
> $$c\left[(n/2+1) + n/2 + (n/2-1) + \ldots + 2 + 1\right]$$
>
> The sum of the first k integers is $k(k+1)/2$. Here we have the sum of the first $k = (n/2+1)$ integers. Therefore, the total is:
>
> $$c \cdot \frac{(n/2+1)(n/2+2)}{2} = \Theta(n^2).$$
>
> We haven't yet considered the calls to `right.pop(0)`, but they will just double the total time found above. And since $2 \cdot \Theta(n^2)$ is still $\Theta(n^2)$, we see that the overall time taken is $\Theta(n^2)$.

c) What is the time complexity of a `mergesort` which uses the above version of `merge` instead of the linear time version used in lecture? Show your work by showing the new recurrence relation and solving it.

Solution: Recall that the recurrence describing the original version of `mergesort` was $T(n) = 2T(n/2) + \Theta(n)$. There, the $\Theta(n)$ came from the call to `merge`. In the new version of `mergesort`, the $\Theta(n)$ is replaced by $\Theta(n^2)$. Hence we solve the recurrence relation:

$$T(n) = \begin{cases} 2T(n/2) + n^2, & n > 1, \\ 1, & \text{otherwise} \end{cases}$$

We solve this by unrolling. We have $T(n/2) = 2T(n/4) + (n/2)^2 = 2T(n/4) + n^2/4$. Making this substitution:

$$\begin{aligned} T(n) &= 2T(n/2) + n^2 \\ &= 2\left[2T(n/4) + n^2/4\right] + n^2 \\ &= 4 \cdot T(n/4) + n^2/2 + n^2 \\ &= 4 \cdot T(n/4) + \frac{3n^2}{2} \end{aligned}$$

Unrolling again, we have $T(n/4) = 2T(n/8) + n^2/16$. Hence:

$$\begin{aligned} &= 4 \cdot \left[2T(n/8) + n^2/16\right] + \frac{3n^2}{2} \\ &= 8T(n/8) + n^2/4 + \frac{3n^2}{2} \\ &= 8T(n/8) + \frac{7n^2}{4} \end{aligned}$$

To summarize, we have:

Unroll #	Formula
1	$T(n) = 2T(n/2) + n^2$
2	$T(n) = 4T(n/4) + \frac{3n^2}{2}$
3	$T(n) = 8T(n/8) + \frac{7n^2}{4}$

We propose the following general formula. On the kth unroll:

$$T(n) = 2^k T(n/2^k) + \frac{2^k - 1}{2^{k-1}} n^2$$

This process will continue until we have $n/2^k = 1$, that is, $\log_2 n$ times. Using this value of k in our general formula, we obtain:

$$\begin{aligned} T(n) &= 2^{\log_2 n} T(n/2^{\log_2 n}) + \frac{2^{\log_2 n} - 1}{2^{(\log_2 n)-1}} n^2 \\ &= nT(n/n) + \frac{n-1}{n/2} n^2 \\ &= n \cdot 1 + \Theta(1) \cdot n^2 \\ &= \Theta(n^2) \end{aligned}$$

Hence the time complexity of this new algorithm is $\Theta(n^2)$. This is much worse than the time complexity of a well-implemented `mergesort`, which is $\Theta(n \log n)$.

Programming Problem 1.

Suppose you are trying to remove outliers from a data set consisting of points in $\mathbb{R}^d$. One of the simplest approaches is to remove points that are in "sparse" regions – that is, points that don't have many other points close by. To do this, we might calculate the distance from a point to it's kth closest neighbor. If this distance is above some threshold, we consider the point an outlier.

More generally, the task of finding the distance from a query point to its kth closest "neighbor" is a common one in data science and machine learning. Here, we'll consider the 1-dimensional version of the problem of finding kth neighbor distance. In a file named `knn_distance.py`, write a function named `knn_distance(arr, q, k)` that returns a pair of two things:

- the distance between `q` and the kth closest point to `q` in `arr`;
- the kth closest point to `q` in `arr` itself

The query point `q` does not need to be in `arr`. For simplicity, `arr` will be a Python list of numbers, and `q` will be a number. `k` should start counting at one, so that `knn_distance(arr, q, 1)` returns the distance between `q` and the point in `arr` closest to `q`. Your approach should have an expected time of $\Theta(n)$, where n is the size of the input list. Your function may modify `arr`. In cases of a tie, the point you return is arbitrary (though the distance is not). Your code can assume that k will be $\leq$ `len(arr)`.

Example:

```
>>> knn_distance([3, 10, 52, 15], 19, 1)
(4, 15)
>>> knn_distance([3, 10, 52, 15,], 19, 2)
(9, 10)
>>> knn_distance([3, 10, 52, 15], 19, 3)
(16, 3)
```

As this is a programming problem, submit your code to the Gradescope autograder.

Solution: The idea is to compute the distance from `q` to all of the points in `arr` in $\Theta(n)$ time, then use quickselect to find the kth order statistic in $\Theta(n)$ expected time. The tricky part is recovering the kth point from the kth distance. You could loop back through the distances searching for the index at which the kth distance occurs, then use this to index into `arr`; essentially, a linear search. This would still be linear time, but there's an approach that you might consider a little cleaner: instead of doing quickselect on the distances alone, run quickselect on tuples of distance/point pairs. This solution is shown below: